🏛 GLA UNIVERSITY, MATHURA — DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

HPC Masterclass | Day 4 Post-Assessment Question Paper

Course: B.Tech CSE 7th Sem — High Performance Computing Masterclass
Module: Day 4 Mastery — Distributed AI Training, Intel VTune TMAM Diagnostics & Cluster Optimization
Duration: 60 Minutes | **Total Questions**: 50 MCQs | **Maximum Marks**: 50

Student Name: _____ **Roll Number**: _____ **Section**: _____

📝 Instructions for Candidates

1. All questions are compulsory. Each question carries exactly 1 mark.
2. Select the single best answer for each question.
3. No negative marking.

Q01. What is the key technological advantage of Precise Event-Based Sampling (PEBS) in Intel hardware performance monitoring compared to legacy interrupt-based sampling?

- ▢ **(a)** PEBS allows profiling code without having an operating system installed.
- ▢ **(b)** PEBS captures the exact instruction pointer (IP), registers, and memory linear address directly into a dedicated hardware buffer upon counter overflow, eliminating interrupt skid and attribution error.
- ▢ **(c)** PEBS automatically increases CPU clock frequency by 50%.
- ▢ **(d)** PEBS deletes slow instructions from the executable binary.

Q02. What occurs when the number of microarchitectural hardware events requested in a VTune custom analysis exceeds the number of physical PMU counter registers available per CPU core?

- ▢ **(a)** The processor halts with a machine check exception (MCE).
- ▢ **(b)** Event Multiplexing: VTune time-slices the physical counters among the requested events during runtime and statistically scales the sampled counts to estimate total occurrences.
- ▢ **(c)** VTune cancels the profiling session and deletes the output directory.
- ▢ **(d)** The application is forced to run on a single thread.

Q03. In Linux HPC cluster environments, what is the fundamental difference between VTune's driverless (Linux `perf`) mode and kernel driver (`sep5.ko` / `vtunedrv`) mode?

- ▢ **(a)** Driverless mode only profiles Python scripts, while kernel driver mode only profiles C++.
- ▢ **(b)** Driverless mode utilizes standard Linux `perf_event_open` syscalls without requiring root kernel modules, while the kernel driver mode grants direct low-level PMU register access, uncore event tracking, and advanced PEBS features.
- ▢ **(c)** Driverless mode cannot measure CPU execution time.

- ▢ **(d)** Kernel driver mode requires rebooting the server after every profiling run.

Q04. Why is compiling with '-fno-omit-frame-pointer' strongly recommended for accurate call-stack collection in VTune, even in optimized release builds (-O3)?

- ▢ **(a)** It makes the compiled binary 10 times smaller on disk.
- ▢ **(b)** Preserving the base pointer (RBP register) allows the profiler sampling interrupt to unwind user call stacks deterministically with near-zero runtime overhead, avoiding expensive DWARF debug table parsing.
- ▢ **(c)** It forces the compiler to inline every function.
- ▢ **(d)** It enables hardware floating-point acceleration.

Q05. What does a high ratio of 'CPU Time' to 'Wait Time' in an Intel VTune Threading Analysis profile indicate about an OpenMP workload?

- ▢ **(a)** Threads spend most of their time idle waiting for mutexes, barriers, or work queues.
- ▢ **(b)** Threads are actively executing instructions on assigned CPU cores with high parallel concurrency and minimal synchronization stall overhead.
- ▢ **(c)** The operating system is thrashing swap memory.
- ▢ **(d)** The system clock is running backwards.

Q06. Which specific hardware PMU event counter does Intel VTune use to measure the exact number of clock cycles a core is not halted in power-saving states?

- ▢ **(a)** CPU_CLK_UNHALTED.THREAD
- ▢ **(b)** INST_RETIRED.ANY
- ▢ **(c)** BR_MISP_RETIRED.ALL_BRANCHES
- ▢ **(d)** MEM_LOAD_RETIRED.L3_MISS

Q07. In VTune GPU Offload analysis, what does an engineer examine to determine whether a SYCL/DPC++ kernel is compute-bound or memory-bound on an Intel Data Center GPU Max?

- ▢ **(a)** The color of the GPU heat sink in the datacenter.
- ▢ **(b)** Execution Unit (EU) active occupancy percentage versus GPU memory read/write bandwidth utilization (HBM2e throughput).
- ▢ **(c)** The length of the SYCL kernel function name.
- ▢ **(d)** The user's Linux terminal shell prompt style.

Q08. How can an HPC developer configure VTune to profile only a specific computation phase of an application and ignore long startup/dataset loading phases?

- ▢ **(a)** By using the Intel Instrumentation and Tracing Technology (ITT) API: calling `__itt_resume()` before the phase and `__itt_pause()` after.
- ▢ **(b)** By physically unplugging the keyboard during data loading.
- ▢ **(c)** By deleting the first 100 lines of the program.
- ▢ **(d)** By running the application inside an unmanaged virtual machine.

Q09. What information does setting the environment variable 'INTEL_LIBITTNOTIFY64=1' verify during VTune collection?

- ▢ **(a)** It checks whether the Linux graphics driver supports 4K displays.
- ▢ **(b)** It confirms that the 64-bit ITT notification dynamic library is correctly loaded and receiving API markers from user code.
- ▢ **(c)** It forces all 32-bit floats into 64-bit doubles.
- ▢ **(d)** It prints the supercomputer electricity consumption.

Q10. Which VTune CLI command exports a summary report of a completed analysis in CSV format for automated parsing by CI/CD scripts?

- ▢ **(a)** vtune -export-all -type=text
- ▢ **(b)** vtune -report summary -result-dir ./my_result -format csv -csv-delimiter comma -report-output ./summary.csv
- ▢ **(c)** vtune --make-csv ./my_result
- ▢ **(d)** vtune -save ./my_result.csv

Q11. In the Top-Down Microarchitecture Analysis Method (TMAM), how is the total pool of 'Pipeline Slots' mathematically defined for a multi-core processor?

- ▢ **(a)** Pipeline Slots = Total RAM (GB) / Clock Speed (GHz)
- ▢ **(b)** Pipeline Slots = CPU_CLK_UNHALTED.THREAD x Pipeline Width (e.g., 4 to 6 uOps per cycle on modern Intel microarchitectures)
- ▢ **(c)** Pipeline Slots = Number of Threads x Number of Files
- ▢ **(d)** Pipeline Slots = Total Network Packets Sent / Round Trip Time

Q12. If an Intel VTune TMAM profile reports 'Retiring: 18.2%, Bad Speculation: 4.1%, Front-End Bound: 14.3%, Back-End Bound: 63.4%', what is the dominant performance bottleneck?

- ▢ **(a)** Branch predictor accuracy.
- ▢ **(b)** Instruction fetch and decode throughput.
- ▢ **(c)** Back-End execution stalling (either memory access latency or execution core saturation).
- ▢ **(d)** Operating system process switching.

Q13. What is the primary difference between 'Front-End Latency' and 'Front-End Bandwidth' stalls in TMAM Level 2 analysis?

- ▢ **(a)** Latency applies to integer math; bandwidth applies to floating-point math.
- ▢ **(b)** Front-End Latency occurs when 0 uOps are delivered per cycle (due to i-cache miss, ITLB miss, or branch recovery); Front-End Bandwidth occurs when uOps are delivered, but fewer than the maximum pipeline width (e.g., 2 instead of 6 uOps) due to decode limitations.
- ▢ **(c)** Front-End Bandwidth is measured in megabytes per second; Latency is measured in miles.
- ▢ **(d)** Latency is caused by hard drives; Bandwidth is caused by RAM.

Q14. What microarchitectural flaw triggers a 'Machine Clear' in an Intel processor, causing a spike in Bad Speculation?

- ▢ **(a)** Memory ordering violations (e.g., speculative loads violating consistency models) or self-modifying code, requiring the processor to wipe all in-flight instructions in the Reorder Buffer.
- ▢ **(b)** Turning off the power switch on the server rack.
- ▢ **(c)** Deleting a Python virtual environment.
- ▢ **(d)** Clearing the terminal screen with the 'clear' command.

Q15. In TMAM Level 2 'Memory Bound' analysis, what metric measures execution stalls caused specifically by L3 Last Level Cache misses that must fetch data from off-chip DRAM?

- ▢ **(a)** L1D Bound
- ▢ **(b)** L2 Bound
- ▢ **(c)** DRAM Bound
- ▢ **(d)** Store Forwarding Bound

Q16. What is a 'Store Forwarding Block' detected by VTune Memory Access analysis, and how can it be resolved?

- ▢ **(a)** A block on a hard drive that cannot be formatted.
- ▢ **(b)** A load instruction attempts to read from a memory location written by a prior store, but the addresses overlap partially or data sizes mismatch, preventing hardware forwarding and forcing a stall until the store writes to L1.
- ▢ **(c)** A network firewall blocking port 22.
- ▢ **(d)** A compiler refusing to compile C++ templates.

Q17. In a dual-socket server, why does a high 'Remote DRAM Access' percentage in VTune Memory Bound analysis degrade application scaling?

- ▢ **(a)** Remote DRAM requires manual approval by the systems administrator.
- ▢ **(b)** Cross-socket memory requests must travel across Intel Ultra Path Interconnect (UPI) links, incurring a 50-80% latency penalty and creating inter-socket interconnect contention.
- ▢ **(c)** Remote DRAM automatically encrypts data with 4096-bit keys.
- ▢ **(d)** Remote DRAM access is disabled on Linux.

Q18. How does 'Core Bound -> Divider Contention' in VTune indicate that an inner mathematical loop needs refactoring?

- ▢ **(a)** The processor is dividing by zero and crashing.
- ▢ **(b)** The hardware floating-point division unit is non-pipelined or has high latency (e.g., 10-20 cycles for FP64 division), and loop iterations are issuing repeated divisions instead of multiplying by precomputed reciprocals.
- ▢ **(c)** The CPU core is divided in half physically.
- ▢ **(d)** The network switch is splitting data packets into fragments.

Q19. According to the Intel TMAM optimization hierarchy, which performance bottleneck should an HPC engineer resolve FIRST?

- ▢ **(a)** Fine-tuning vector loop unrolling (Core Bound).

- ▢ **(b)** Eliminating catastrophic Bad Speculation (e.g., high branch mispredictions) and Front-End Latency stalls, because speculative and front-end stalls invalidate downstream back-end analysis.
- ▢ **(c)** Replacing 64-bit doubles with 16-bit floats.
- ▢ **(d)** Changing the color scheme of the terminal.

Q20. What software restructuring technique eliminates 'L1 Data Cache Bound' stalls caused by frequent pointer-chasing in linked lists or trees?

- ▢ **(a)** Switching from pointer-based nodes to flat contiguous arrays (e.g., Structure of Arrays or contiguous flat memory buffers) to exploit hardware cache line loading and spatial prefetchers.
- ▢ **(b)** Adding more pointers to each node.
- ▢ **(c)** Decreasing the size of the CPU L1 cache in BIOS.
- ▢ **(d)** Converting C++ code into Python.

Q21. In the Roofline Model, how is the upper compute ceiling (horizontal line) and memory bandwidth ceiling (slanted line) determined for a given processor?

- ▢ **(a)** By measuring the speed of optical fiber cables.
- ▢ **(b)** Compute ceiling is the processor's Peak Theoretical Floating-Point Performance (GFLOP/s = Cores x Clock x Vector Width x FMAs/cycle); Bandwidth ceiling is Peak Theoretical Memory Bandwidth (GB/s = Channels x Data Rate x Bus Width).
- ▢ **(c)** By running a web browser speed test.
- ▢ **(d)** By inspecting the hard drive transfer rate.

Q22. What is the 'Ridge Point' of an HPC node equipped with dual Intel Xeon Platinum 8480+ processors capable of 10,000 GFLOP/s FP32 and an 8-channel DDR5 memory bandwidth of 307.2 GB/s?

- ▢ **(a)** 0.03 FLOPs/Byte
- ▢ **(b)** 32.55 FLOPs/Byte
- ▢ **(c)** 307.2 FLOPs/Byte
- ▢ **(d)** 10,000 FLOPs/Byte

Q23. If Intel Advisor profiles a convolutional layer and places it on the diagonal slope beneath the DRAM bandwidth roofline, what optimization MUST be prioritized to increase performance?

- ▢ **(a)** Increasing arithmetic intensity: applying operator fusion (e.g., Conv+ReLU+Add) or cache blocking to shift the operational point to the right toward the compute ceiling.
- ▢ **(b)** Adding more CPU cores to the Slurm batch script.
- ▢ **(c)** Increasing the learning rate of the optimizer.
- ▢ **(d)** Recompiling with -O0 to disable optimizations.

Q24. When designing a cache-blocked matrix multiplication kernel ($C = A \times B$) in C++, how should the block size B be calculated to fit in L1 Data Cache (e.g., 32 KB)?

- ▢ **(a)** $B = 1,000,000$
- ▢ **(b)** Three sub-blocks of size $B \times B$ (for A, B, C) must fit in L1: $3 \times B^2 \times \text{sizeof}(float) \leq 32,768$ bytes, which yields $B \approx 52$ (typically chosen as 32 or 48 for vector alignment).

- ▢ **(c)** B should always equal the total number of threads.
- ▢ **(d)** $B = \text{sizeof}(\text{pointer})$

Q25. What is the penalty of executing unaligned AVX-512 vector load instructions (`_mm512_loadu_ps`) across a 64-byte cache line boundary?

- ▢ **(a)** The processor emits a hardware fault and terminates.
- ▢ **(b)** A Split Load: accessing data that straddles two distinct 64-byte cache lines requires two separate cache lookups, two memory accesses, and instruction pipeline serialization, doubling load latency.
- ▢ **(c)** The CPU frequency drops to 1 GHz.
- ▢ **(d)** Data is rounded to the nearest integer.

Q26. Which C/C++ memory allocation function guarantees that dynamically allocated arrays are aligned to 64-byte hardware cache boundaries for AVX-512 SIMD operations?

- ▢ **(a)** malloc(size)
- ▢ **(b)** posix_memalign(&ptr, 64, size) or _mm_malloc(size, 64)
- ▢ **(c)** calloc(1, size)
- ▢ **(d)** realloc(ptr, size)

Q27. Why is the 'Structure of Arrays' (SoA) memory layout superior to 'Array of Structures' (AoS) for SIMD vectorization in deep learning and HPC codes?

- ▢ **(a)** SoA makes code shorter to type.
- ▢ **(b)** SoA stores identical fields contiguously in memory (e.g., all X coordinates together), allowing a single 512-bit vector instruction to load 16 consecutive values directly into an AVX-512 register without strided gather overhead.
- ▢ **(c)** AoS cannot be compiled on Linux.
- ▢ **(d)** SoA deletes unused variables automatically.

Q28. What are 'Non-Temporal Streaming Stores' (e.g., `_mm512_stream_ps` / `MOVNTPS`), and when should an HPC engineer use them?

- ▢ **(a)** Stores that write data to an external tape drive.
- ▢ **(b)** Store instructions that write directly to DRAM, bypassing CPU caches and avoiding the 'write-allocate' penalty when writing large destination buffers that will not be read again soon, preserving cache residency for other data.
- ▢ **(c)** Stores that execute only at night.
- ▢ **(d)** Stores that encrypt memory contents.

Q29. Why does a vector loop with a small trip count (e.g., looping 17 times on an AVX-512 processor) often exhibit poor vectorization efficiency?

- ▢ **(a)** AVX-512 cannot process odd numbers.
- ▢ **(b)** AVX-512 processes 16 floats per vector step; 17 iterations requires 1 full vector step and 1 scalar remainder step, meaning loop startup, remainder masking overhead, and cleanup dominate execution time.
- ▢ **(c)** The compiler refuses to vectorize loops smaller than 100 iterations.
- ▢ **(d)** The processor clock halts during remainder loops.

Q30. How does transitioning an algorithm from IEEE FP64 (double precision) to Bfloat16 (BF16) affect its position in the Roofline Model?

- ▢ **(a)** It halves arithmetic intensity.
- ▢ **(b)** It doubles arithmetic intensity (halving memory footprint from 8 bytes to 2 bytes per operand) and raises the compute ceiling by $4\times$ to $8\times$ via specialized hardware matrix units (e.g., Intel AMX).
- ▢ **(c)** It has zero effect on the Roofline chart.
- ▢ **(d)** It moves the operational point off the chart.

Q31. In an enterprise multi-node Slurm job script for 4 nodes with 4 GPUs each, why is setting '#SBATCH --cpus-per-task=8' essential?

- ▢ **(a)** To allow the Slurm scheduler to run 8 copies of the bash shell.
- ▢ **(b)** To allocate 8 dedicated physical CPU cores per distributed worker process, ensuring sufficient host threads for asynchronous data loading, JPEG decompression, and data augmentation in `tf.data` without starving GPUs.
- ▢ **(c)** Because Slurm will reject any script where CPUs per task is less than 8.
- ▢ **(d)** To force the GPUs to execute 8 threads simultaneously.

Q32. Why is the '#SBATCH --exclusive' flag mandatory when running reproducible distributed deep learning benchmarks on shared HPC clusters?

- ▢ **(a)** To ensure that no other user jobs run on the allocated nodes, eliminating CPU core preemption, cache pollution, memory bus contention, and PCIe/NIC sharing jitter.
- ▢ **(b)** To ensure that only the superuser can view job output files.
- ▢ **(c)** To run the job exclusively on node 0 of the cluster.
- ▢ **(d)** To permanently prevent other users from logging into the cluster.

Q33. What does the Slurm command 'scontrol show hostnames $SLURM_JOB_NODELIST' output when executed on an allocation of 'node[01-02,05]'?

- ▢ **(a)** node[01-02,05]
- ▢ **(b)** node01 node02 node05
- ▢ **(c)** A list of IP addresses from external DNS.
- ▢ **(d)** The total number of CPU sockets in the cluster.

Q34. In a distributed Slurm environment, what does the option '--cpu-bind=cores' enforce on spawned tasks?

- ▢ **(a)** It glues the CPU chips to the motherboard socket.
- ▢ **(b)** It binds each spawned process and its child threads strictly to its allocated physical CPU cores using Linux CPU affinity masks, preventing task migration and preserving cache locality.
- ▢ **(c)** It forces all tasks to execute on CPU core 0.
- ▢ **(d)** It disables multi-core processing.

Q35. How does an HPC engineer inspect the historical peak memory usage and CPU efficiency of a completed distributed job with Job ID 124589?

- ▢ **(a)** squeue -j 124589
- ▢ **(b)** sacct -j 124589 --format=JobID,JobName,AllocCPUS,Elapsed,MaxRSS,ExitCode

- ▢ **(c)** sinfo -N 124589
- ▢ **(d)** scancel 124589

Q36. What is the function of a Slurm 'Epilog' script configured on compute nodes by cluster administrators?

- ▢ **(a)** Writing introductory remarks for research papers.
- ▢ **(b)** Executing with root privileges upon job termination to clean up node state: killing orphaned user processes, releasing locked GPU memory, and sanitizing `/tmp` scratch partitions.
- ▢ **(c)** Printing a welcome message on user SSH login.
- ▢ **(d)** Backing up node BIOS firmware to disk.

Q37. In Slurm multifactor job scheduling, what does the 'Fair-Share' priority factor measure?

- ▢ **(a)** How equally the physical servers are divided across student dormitories.
- ▢ **(b)** The difference between an account's allocated compute target and its actual historical resource consumption over an exponential half-life decay period, boosting priority for under-served users.
- ▢ **(c)** The amount of financial tuition paid by the student.
- ▢ **(d)** The total number of git commits submitted by the user.

Q38. What occurs when a multi-node Slurm job exceeds the wall-clock limit specified by '#SBATCH --time=01:00:00'?

- ▢ **(a)** The job is automatically granted 24 more hours.
- ▢ **(b)** Slurm sends a SIGTERM signal to all job processes, followed shortly by SIGKILL, marks the job state as 'TIMEOUT', and immediately revokes node allocation.
- ▢ **(c)** The job pauses until the user logs back in.
- ▢ **(d)** The compute nodes are powered off.

Q39. In a distributed Slurm environment, why is it recommended to launch multi-node MPI/Horovod applications using 'srun' rather than standalone 'mpirun'?

- ▢ **(a)** Because mpirun cannot run C++ programs.
- ▢ **(b)** Because 'srun' directly queries the Slurm job step manager, automatically inheriting node allocations, task counts, CPU affinity masks, and GRES GPU bindings without requiring manual hostfile creation.
- ▢ **(c)** Because srun bypasses Linux user permissions.
- ▢ **(d)** Because mpirun disables high-speed InfiniBand interfaces.

Q40. What Slurm directive requests that tasks within a job be placed on nodes that reside on the same network switch leaf to minimize inter-node network latency?

- ▢ **(a)** #SBATCH --switches=1
- ▢ **(b)** #SBATCH --network=fast
- ▢ **(c)** #SBATCH --topology=flat
- ▢ **(d)** #SBATCH --leaf=true

Q41. What is the total communication volume transferred by each worker node in the Ring-AllReduce algorithm to synchronize gradient tensors of size M bytes across N workers?

- ▢ **(a)** $2 \times \frac{N-1}{N} \times M$
- ▢ **(b)** $N^2 \times M$
- ▢ **(c)** $\frac{M}{N^2}$
- ▢ **(d)** $M \times N$

Q42. Why does Ring-AllReduce scale with high bandwidth efficiency across 64+ worker nodes compared to a Centralized Parameter Server (PS)?

- ▢ **(a)** Because Ring-AllReduce transfers data through wireless routers.
- ▢ **(b)** Because the communication volume per worker node is asymptotically constant ($2M$) and completely independent of the number of nodes N, whereas the Parameter Server network traffic scales linearly ($O(N)$), creating a central network bottleneck.
- ▢ **(c)** Because Ring-AllReduce eliminates the need for gradient computation.
- ▢ **(d)** Because Parameter Servers cannot store floating-point numbers.

Q43. In Horovod distributed deep learning, what does an engineer diagnose when a Chrome tracing profile (`HOROVOD_TIMELINE=timeline.json`) displays long 'NEGOTIATE_ALLREDUCE' intervals?

- ▢ **(a)** The worker nodes are debating which programming language to use.
- ▢ **(b)** Worker processes on different nodes are arriving at the synchronization barrier at disparate times (due to worker desynchronization, stragglers, or I/O stalls), forcing early-arriving ranks to wait before tensor reduction can be collectively negotiated.
- ▢ **(c)** The InfiniBand switch has lost power.
- ▢ **(d)** The model has achieved 100% training accuracy.

Q44. How does 'Tensor Fusion' in Horovod (controlled by `HOROVOD_FUSION_THRESHOLD`) resolve network latency degradation during deep learning training?

- ▢ **(a)** By deleting small neural network layers.
- ▢ **(b)** By packing many small gradient tensors into a single contiguous fusion buffer (e.g., 64 MB) before initiating the collective AllReduce call, amortizing message transmission overhead over high-speed networks.
- ▢ **(c)** By combining multiple GPU cards with soldering wire.
- ▢ **(d)** By converting all weights into single-precision integers.

Q45. What is the purpose of enabling Horovod Autotuning (`export HOROVOD_AUTOTUNE=1`) during distributed training on an HPC cluster?

- ▢ **(a)** It automatically tunes the guitar in the student break room.
- ▢ **(b)** It runs dynamic Bayesian optimization during early training epochs to search for the highest-throughput combination of `HOROVOD_FUSION_THRESHOLD`, cycle time, and response cache settings for the specific cluster interconnect fabric.
- ▢ **(c)** It automatically modifies the neural network hyperparameters.
- ▢ **(d)** It tunes the cooling fan speed in the server room.

Q46. How does 'Communication-Computation Overlap' in distributed training boost overall scaling efficiency?

- ▢ **(a)** By executing training without using a network.
- ▢ **(b)** By triggering the AllReduce gradient synchronization of outer layers immediately in the background while earlier layers are still actively executing their backward pass computations on the accelerator.
- ▢ **(c)** By skipping the backward pass entirely on even-numbered epochs.
- ▢ **(d)** By converting all 32-bit floats into 1-bit booleans.

Q47. In enterprise InfiniBand and RoCE v2 networks, why are 'Jumbo Frames' (MTU = 9000 bytes) configured instead of standard Ethernet MTU (1500 bytes)?

- ▢ **(a)** Because Jumbo frames allow transferring image files without compression.
- ▢ **(b)** Larger payload size reduces the total number of packets by a factor of 6 for a given tensor size, drastically cutting packet header overhead, switch buffer allocations, and packet processing CPU cycles.
- ▢ **(c)** Because standard 1500-byte MTU cannot transmit floating-point data.
- ▢ **(d)** Because Jumbo frames eliminate the need for optical fiber cables.

Q48. What is the primary motivation for 'Gradual Learning Rate Warmup' when scaling distributed deep learning with Horovod across 64+ nodes?

- ▢ **(a)** To allow the GPU silicon to reach its operating temperature.
- ▢ **(b)** Applying the Linear Scaling Rule ($64 \times LR$) immediately from step 0 on randomly initialized weights causes massive gradient variance and loss explosion; gradual warmup ramps the rate linearly over 5 epochs to stabilize early optimization.
- ▢ **(c)** To ensure that all worker nodes establish internet connections.
- ▢ **(d)** Because Slurm terminates jobs that use high learning rates early on.

Q49. Why is checkpoint saving in Horovod strictly gated behind 'if hvd.rank() == 0:' on cluster filesystems?

- ▢ **(a)** Only rank 0 has access to model weights.
- ▢ **(b)** To prevent all N distributed workers from simultaneously writing to the exact same checkpoint file path on the shared parallel filesystem (NFS/Lustre), which causes severe write collisions, lock contention, and file corruption.
- ▢ **(c)** Rank 0 is the only process with a Python interpreter.
- ▢ **(d)** Other ranks will delete the model checkpoint.

Q50. What is the primary indicator that an HPC cluster has a 'Straggler Node' during synchronous distributed training, and how is it identified?

- ▢ **(a)** One node produces text output in reverse order.
- ▢ **(b)** All nodes complete steps in identical time, but training loss increases.
- ▢ **(c)** Profiling traces show that $N - 1$ nodes spend excessive time in `MPI_Barrier` or `WAIT_ALLREDUCE`, while one specific node exhibits prolonged forward/backward compute times or I/O stalls.
- ▢ **(d)** The head node deletes the Slurm batch script.